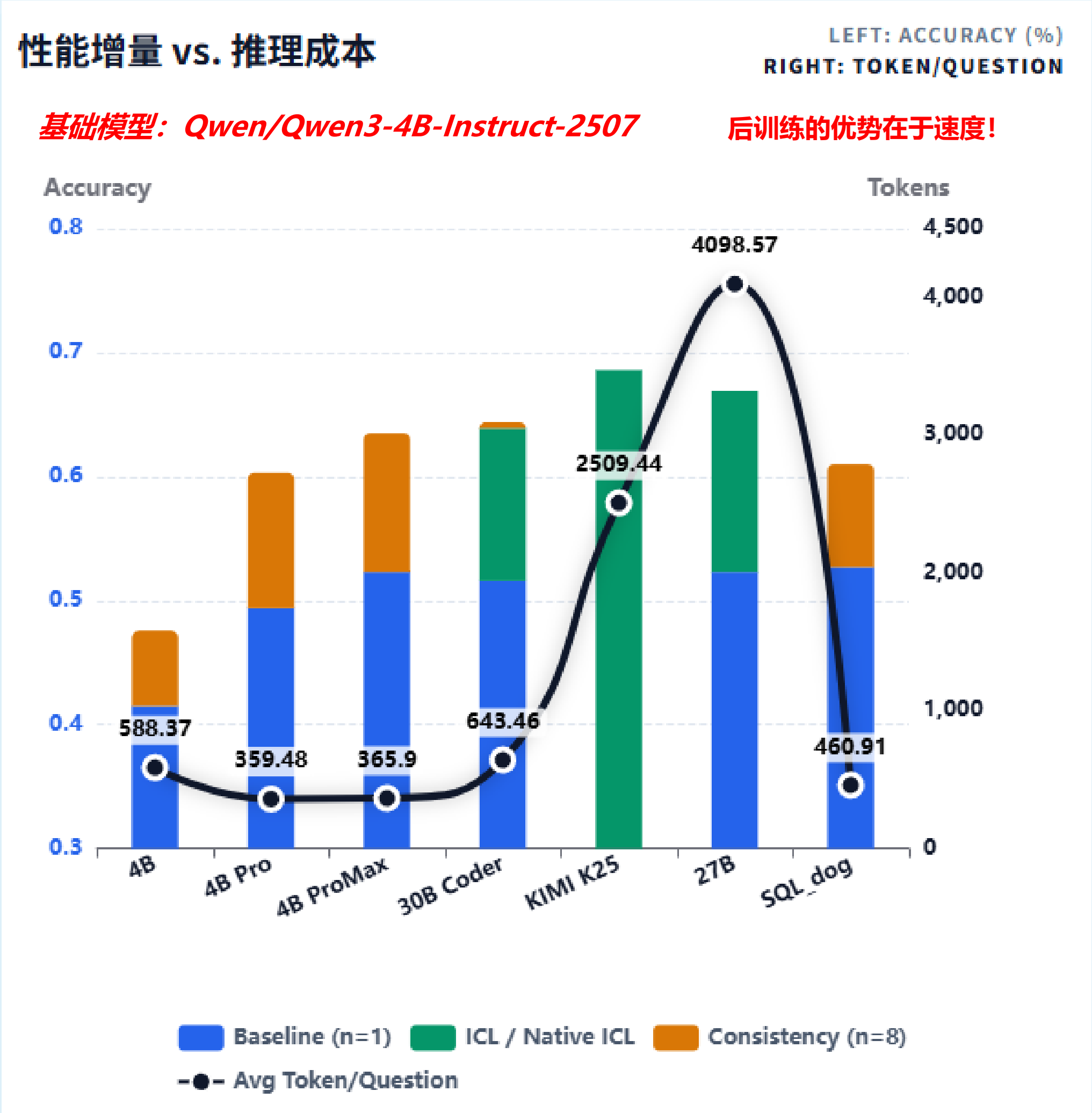
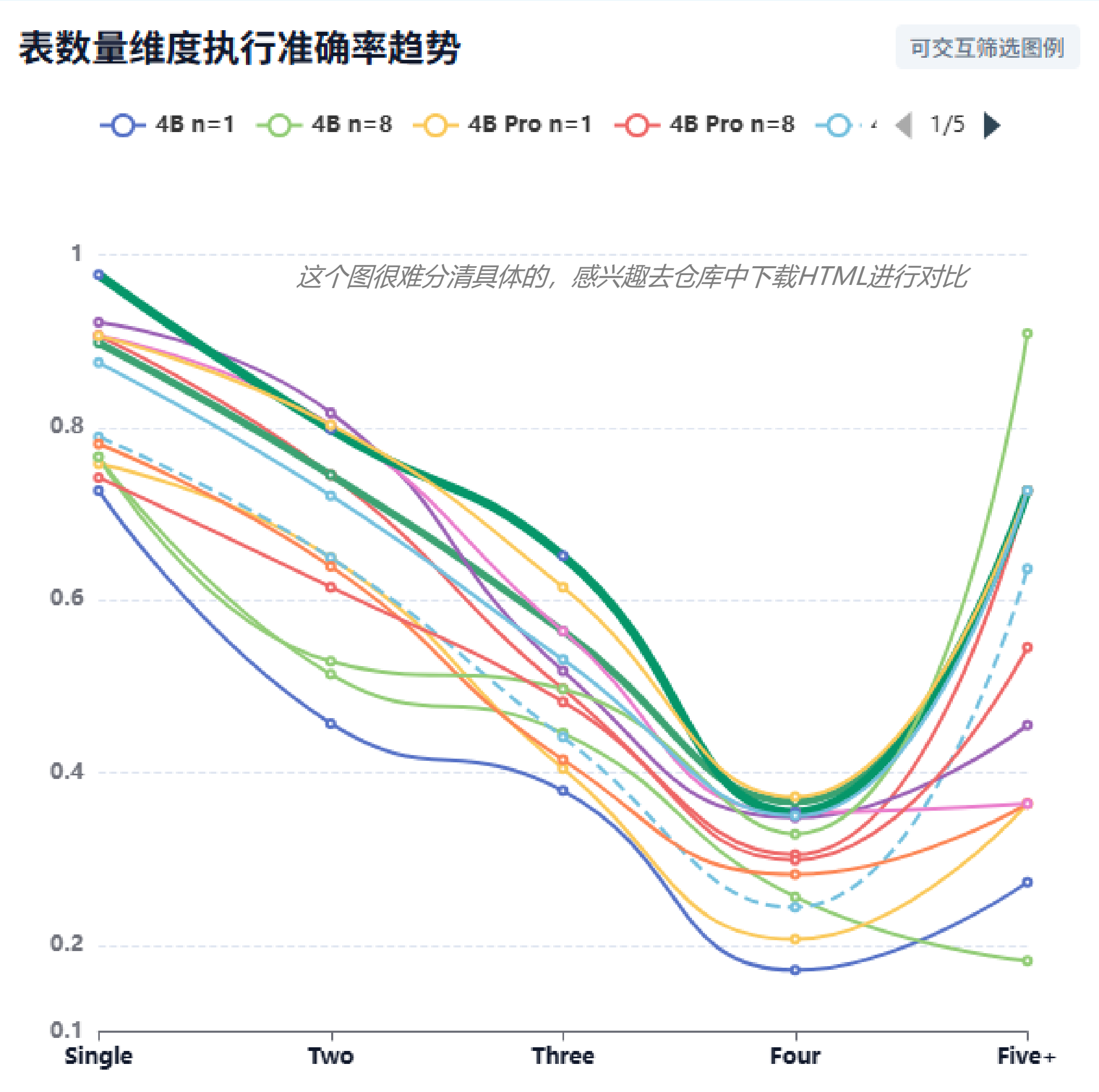


- KIMI K2.5 和 Qwen3.5 27B统一开启思考模式
- SQL_dog(4B)：为后训练的**多轮**Text to SQL问答模型，其对应的论文还要等六月份才online
- Consistency的方法看上去非常有效，因为这个数据集的问题标注出奇的模糊
- ICL：为我随机从训练集中选择的7条（问题，SQL ）对，这有利于LLM了解数据集的特征
- 完整复现对应的仓库能够获得4B Pro的性能
- SQL_dog模型是为了配合接下来开源的一个Text to SQL端到端框架搭建的agent项目
- **本项目的后训练完全基于一张3090来完成，核心是通过蒸馏来提升性能，没有硬核算法。**

Github: qcjySONG/SQL_dog



数据集介绍

- 整体使用mimic_iv这样一个医疗数据库（Sqlite，英文），其拥有17张小表，并且其中不乏一些列名非常奇怪的列，尽管它可能难以和传统企业的上古老数据库匹配，但是其实和一些AI Native数据库比较接近了。（**Text to SQL要是想做就一定要治理数据，你不能要求LLM理解一些老程序员都不理解的企业遗留的上古垃圾数据库**）请分析：dob是什么意思哈哈哈~

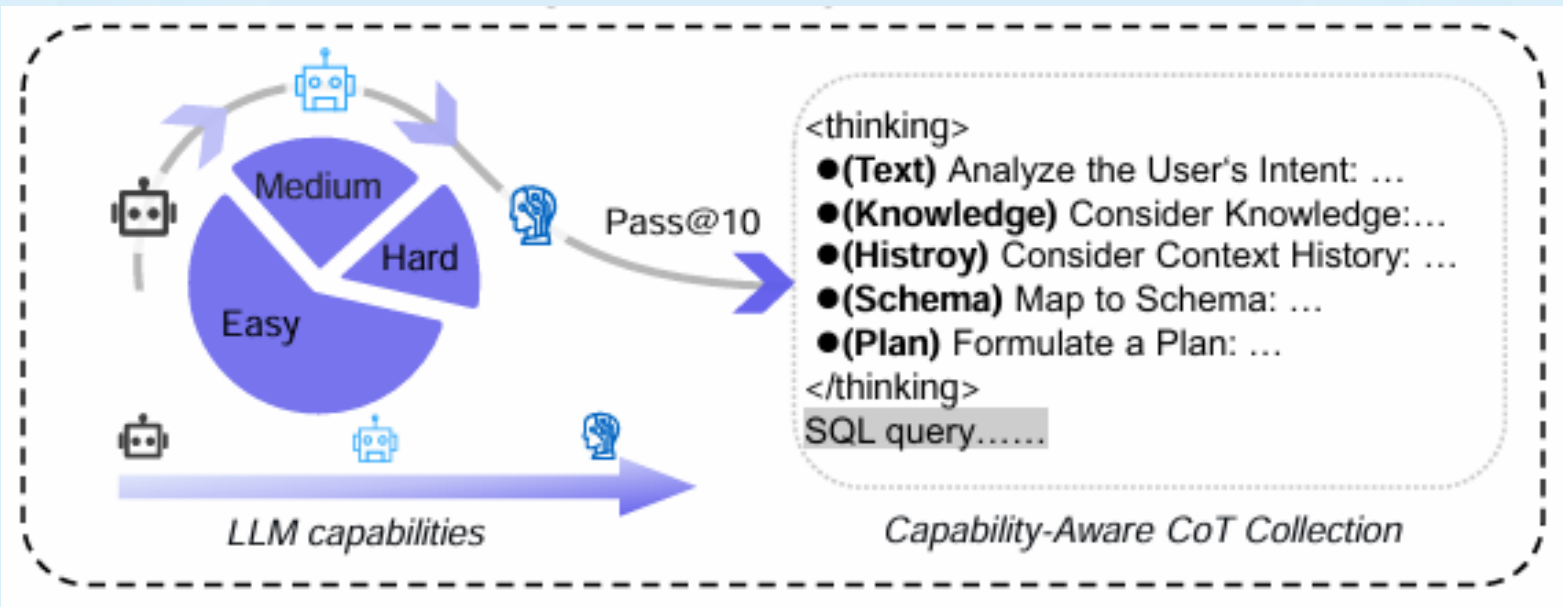
```
CREATE TABLE patients (
  row_id INT, -- example: [0, 1, 2]
  subject_id INT, -- example: [10014729, 10003400, 10002428]
  gender VARCHAR(5), -- example: ["f"]
  dob TIMESTAMP(0), -- example: ["2079-07-22 00:00:00", "2028-07-25 00:00:00", "2020-07-30 00:00:00"]
);
```

- 这个数据集标注很脏，我尽我所有能test里面选择了干净的数据。

Github: qcjySONG/SQL_dog

整体方法

- 我们假设你拥有一个数据库，和接近100条平时对这个数据库进行的查询的SQL。
- **构建（问题，SQL）对：**这一步的构建非常难，它涉及不同数据库函数、值和对应的业务指数如何注入与多轮对话方式。（本仓库使用的是别人构建好的训练集，后面我们会写我在实际场景下构建数据集的方法）
- **结构化CoT蒸馏：**传统的工作基于（问题，SQL）对直接训练，我们这里多一步，通过其它模型蒸馏对应的短思维链，因为SQL可以执行，我们可以很简单的判断蒸馏思维的正确性，*进而判断当前（问题，SQL）构造的合理性。*具体的，我们通过多个（不同能力和价格）模型Pass@10来生成正确的思维链，这一步非常烧钱。



在这里额外插一嘴，此前我被一个应该是***Dify agent 砖家***面试，它因为我蒸馏的模型其实总体是比R1便宜的并且最后效果为什么能够超过R1而质疑我项目的真实性，这个砖家可能连DeepSeek R1原文论文都没读过。

- **SFT**：到这一步，我们已经收集到了大约3500条数据（问题，CoT），实际上这个数据量偏少，难以覆盖一些边边角角。使用Unsloth进行了两轮的Lora微调。
  - 4B Pro：0.494 （+ 自一致性：0.603）
  - 4B Pro Max：0.523（+ 自一致性：0.635）

  值得注意的是Pro Max训练的数据是一样的，仅随机在Prompt中加入一些token（类似）：

  CXX，那么简单都不会做，cxx，仔细看题！

  此外，我还尝试过在SFT后的模型上面DPO（同样能在24GB下跑），确实还是能够加强推理能力的。

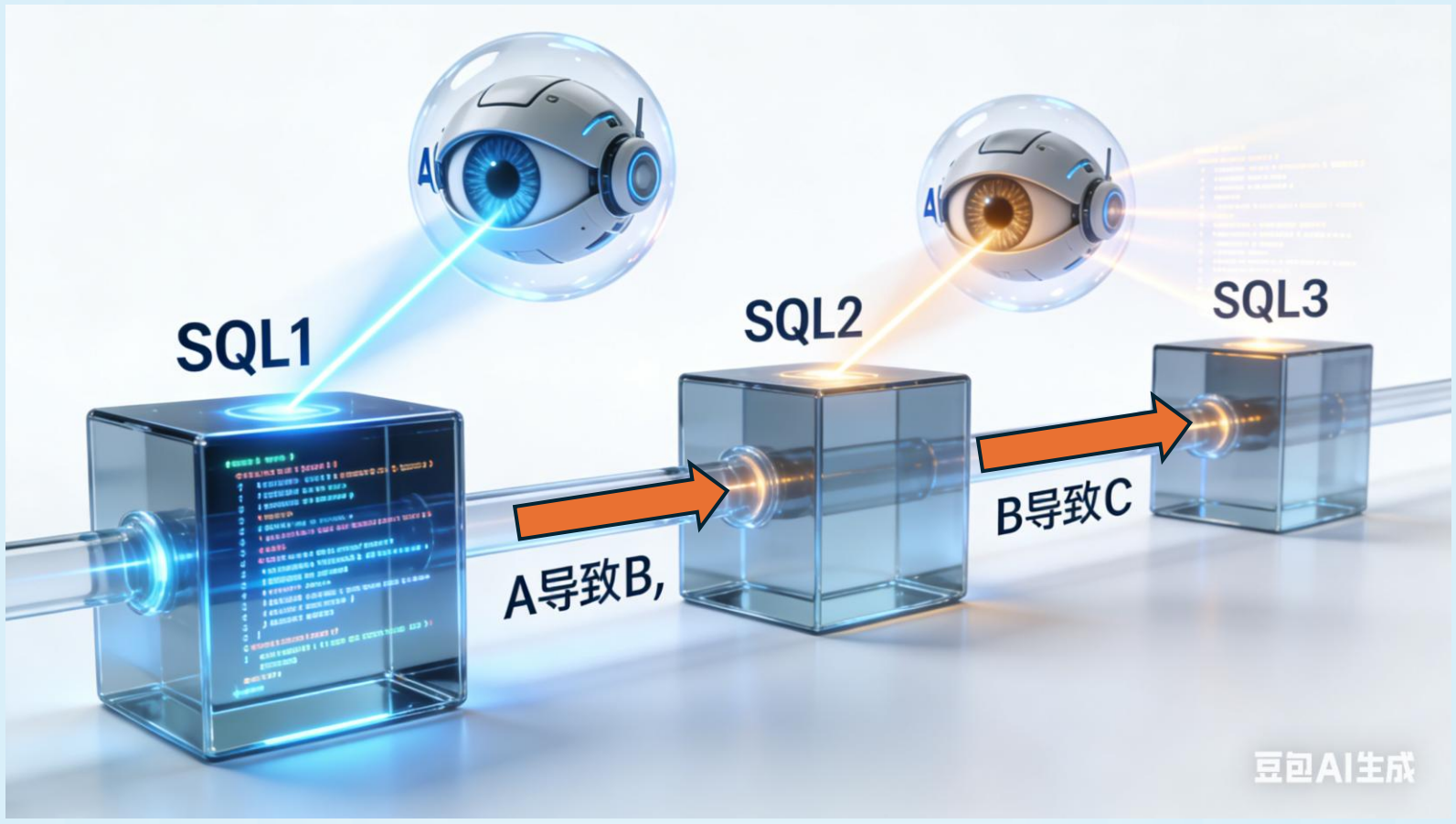
Prompt构建：

- 数据库DDL（包含示例）
- 当前问题
- 数据库值检索
- 外部知识检索的内容（如果有）
- 基础指令

对于NL2SQL，这两部分其实是有必要的，但是我也在想办法和面试官解释其与RAG的区别，因为它其似乎不涉及召回的问题。有的砖家**看了几篇VX RAG介绍文章**不听解释就逮这个问，为什么不Graph Rag，甚至还有怎么切片（数据库值不用切片），最近我将这一步和问题重写揉在一起了，因为我也看了几篇VX，它们好像也有检索然后问题重写的。我也在思考这一步怎么解释。

合成数据（SQL，问题），此部分作为参考，没办法给代码

- **单轮情况下，参考OmniSQL：**它通过**数据库函数、提供的表和数据库值**的不同来构建对应的SQL语句，最后基于SQL骨干去重。
  - 思考：对于一个数据库，它就几个问答的方向，其实不用覆盖全部的SQL函数，比如很少人在SQLite里面用那个正则函数，因此你是需要根据你的业务场景删除部分奇怪的函数来控制合成数据的合理性。
- **多轮场景：**基于启发式or基于LLM导向。
  - 启发式：基于生成的单轮SQL，for循环几个方向提示LLM生成下一条SQL: [增加过滤条件、增加 JOIN、改变聚合粒度、改变排序、增加随机业务逻辑]。这样的方法有点人机，但是其能够覆盖尽可能的方面。
  - LLM控制：让LLM根据此前的所有SQL判定下一轮合成SQL的走向，LLM启发是我用的比较多的手段。



- **合成用户问题：**上面讨论的都是生成SQL，根据SQL生成用户问题是比较简单的了，只要Prompt即可，当然也可以折腾DPO，**合成的过程每条SQL至少要合成两类用户问题**：
  - 标准型：用户问题不偏不倚对应SQL语句
  - 不标准型：至少将实体对应给变换一下，以增强其鲁棒性。
- **对应真实业务：**假设一个数据库至少的有若干条真实的SQL语句！这些真实的SQL语句可以在合成过程中进行Few Shot，这样合成过程就不脱离数据库场景了（即不那么人机了）

Note：SQL_dog就是完全在合成的多轮问答上面进行后训练的。

性价比之王 (EFFICIENCY)	最高成本支出	原生 ICL 标杆	SQL_DOG 表现
4B ProMax	**27B Model**	**KIMI K25**	**61.0% Acc**
365 Tokens @ 63.5%	4,098 Tokens / Q	2,509 Tokens / Q	n=8 模式下

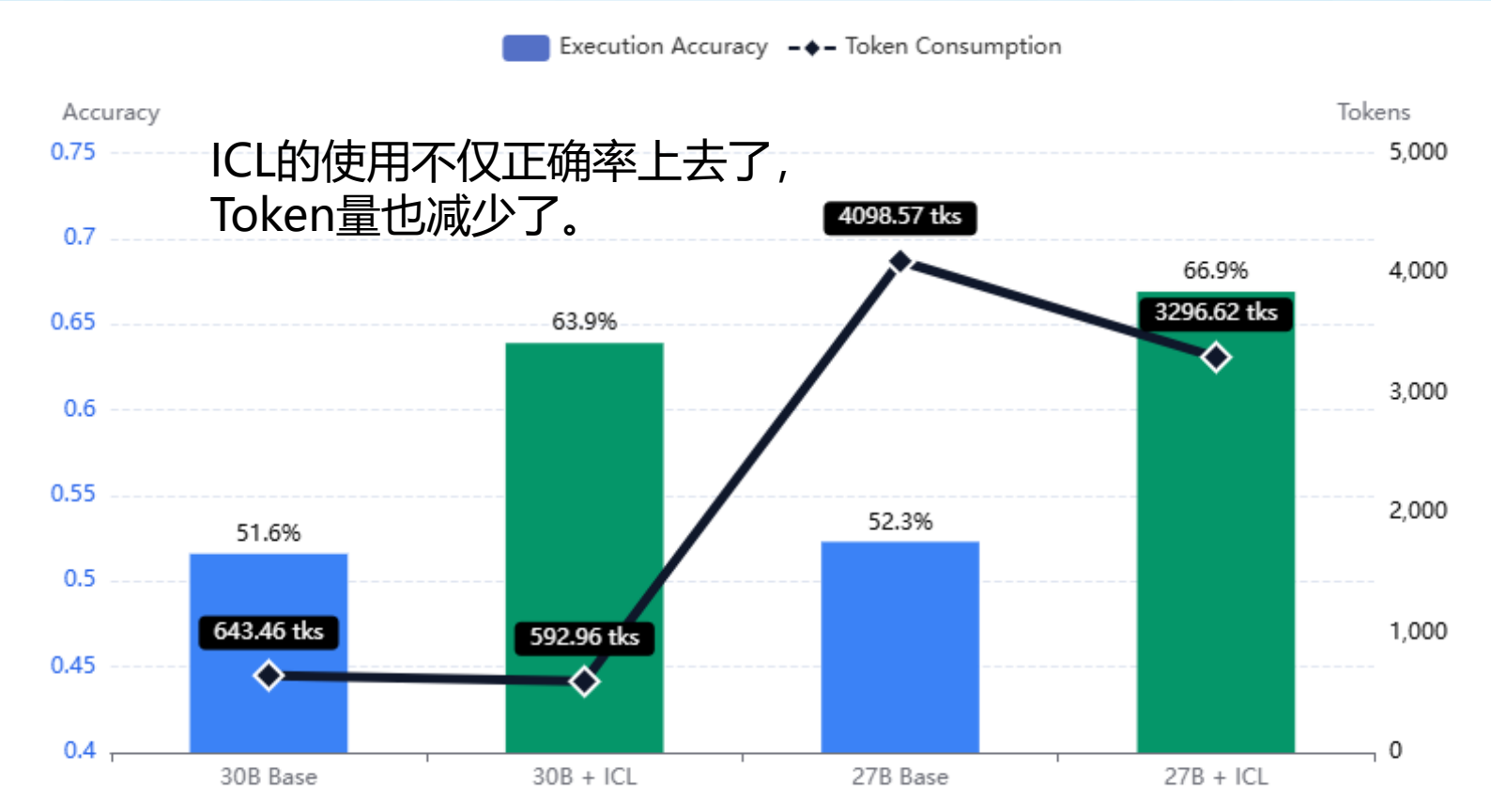
Github: qcjySONG/SQL_dog

Agent时代是否需要后训练?

- **如果你写论文，我建议你后训练，因为纯agent（提示词工程）的论文非常难发了。**
- 请思考：
  - 数据库（业务）是否固定
  - 是否对速度有要求
  - 隐私是否需要保证

- 实际的每个人的工作情况都不一样。此前有个人做Dify套壳的，他们和别人夸下海口说能够100%正确率，**对面居然信了，**然后他们搞不定，将问题抛给我（美其名约请教）：32B、不能后训练、对于延迟有要求，100%正确率。然后我将他们删了。

- 对我的此前的尝试而言，使用稍微强一点的模型（4o），配合上相应的ICL技术（因为一个数据库就那么几个提问的方向）、预定义的若干个插槽，这一套组合拳下来别人已经很高兴了。复杂一点的场景各种tool组合下来效果也不错（当然时间也上去了）

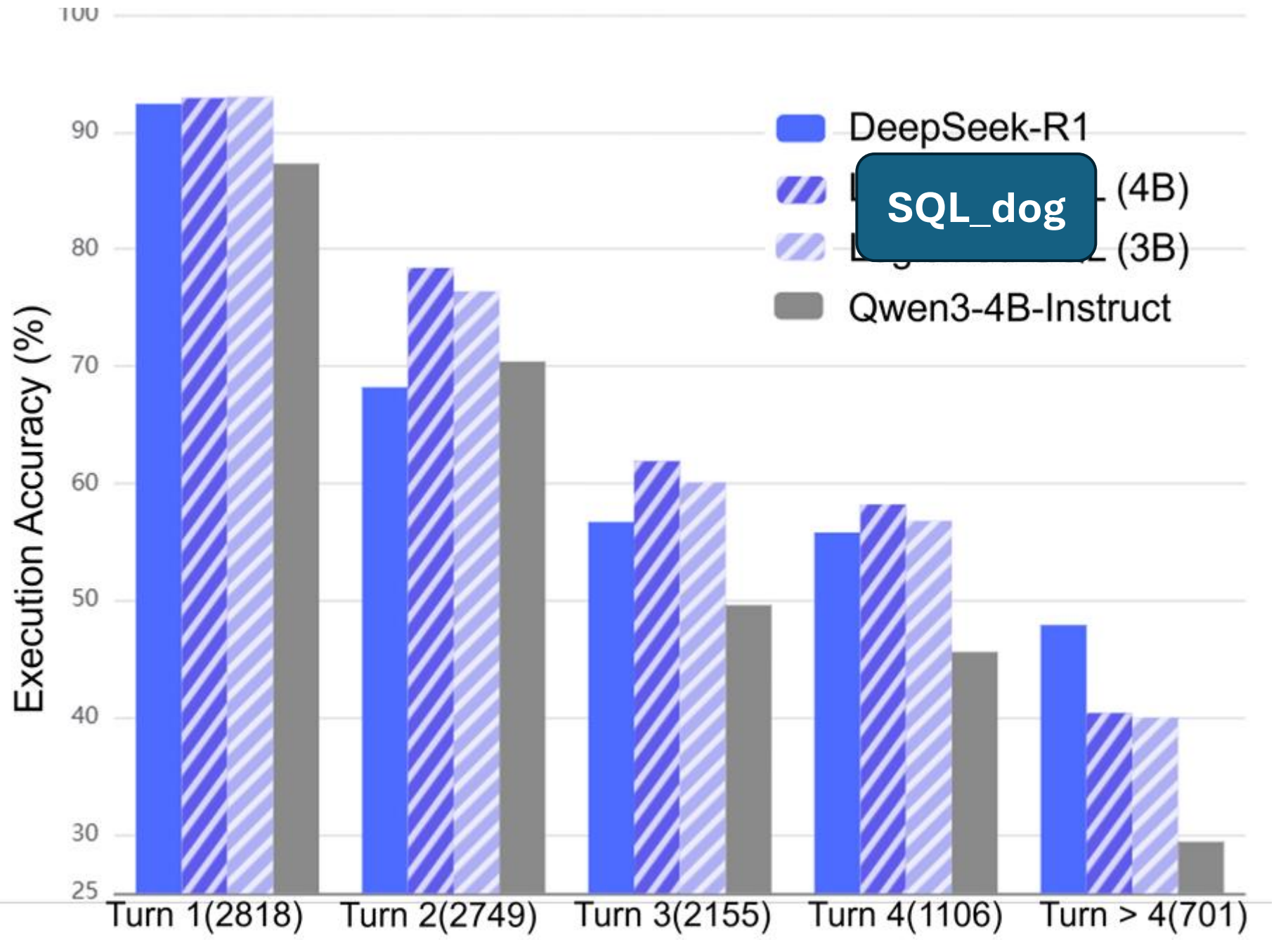


后续训练过程、数据集、模型和一个端到端的对应的Dome级Web都会上传到对应的Github仓库中
如果对你找工作、工作有帮助，请给我一个Start！

Github: qcjySONG/SQL_dog

补充：单轮与多轮对话

- 我下面展示的是SQL_dog对应论文中的现象，在高轮次下简单的SFT后训练似乎无法让其超越R1这种参数怪兽。我的实践下来是一定要使用GRPO对高高轮次的数据集继续训练的，但是这显然不是3090能够承受的范畴了。



总结，这个项目提供了（单轮场景）：

- 一份完整的后训练数据集和流程，并且全部可以在一块24GB显存的显卡上完成，咬咬牙也可以16GB的显卡。
- 这些折腾过程你完全可以写进简历里面，我面试的时候他们就问的比较多这个。
- 一个完整的agent工作流项目，让你可以运用起来训练的模型。
- Free，请不要爬取放到CSDN，哈哈哈。

SQL Dog　　SQL Dog　　SQL Dog

后续训练过程、数据集、模型和一个端到端的对应的Dome级Web都会上传到对应的Github仓库中
如果对你找工作、工作有帮助，请给我一个Start！

Github: qcjySONG/SQL_dog

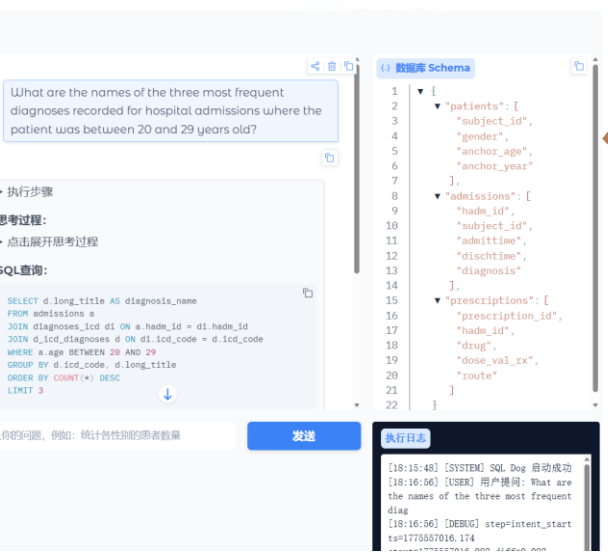
GRPO
DPO
SFT

SQL生成器

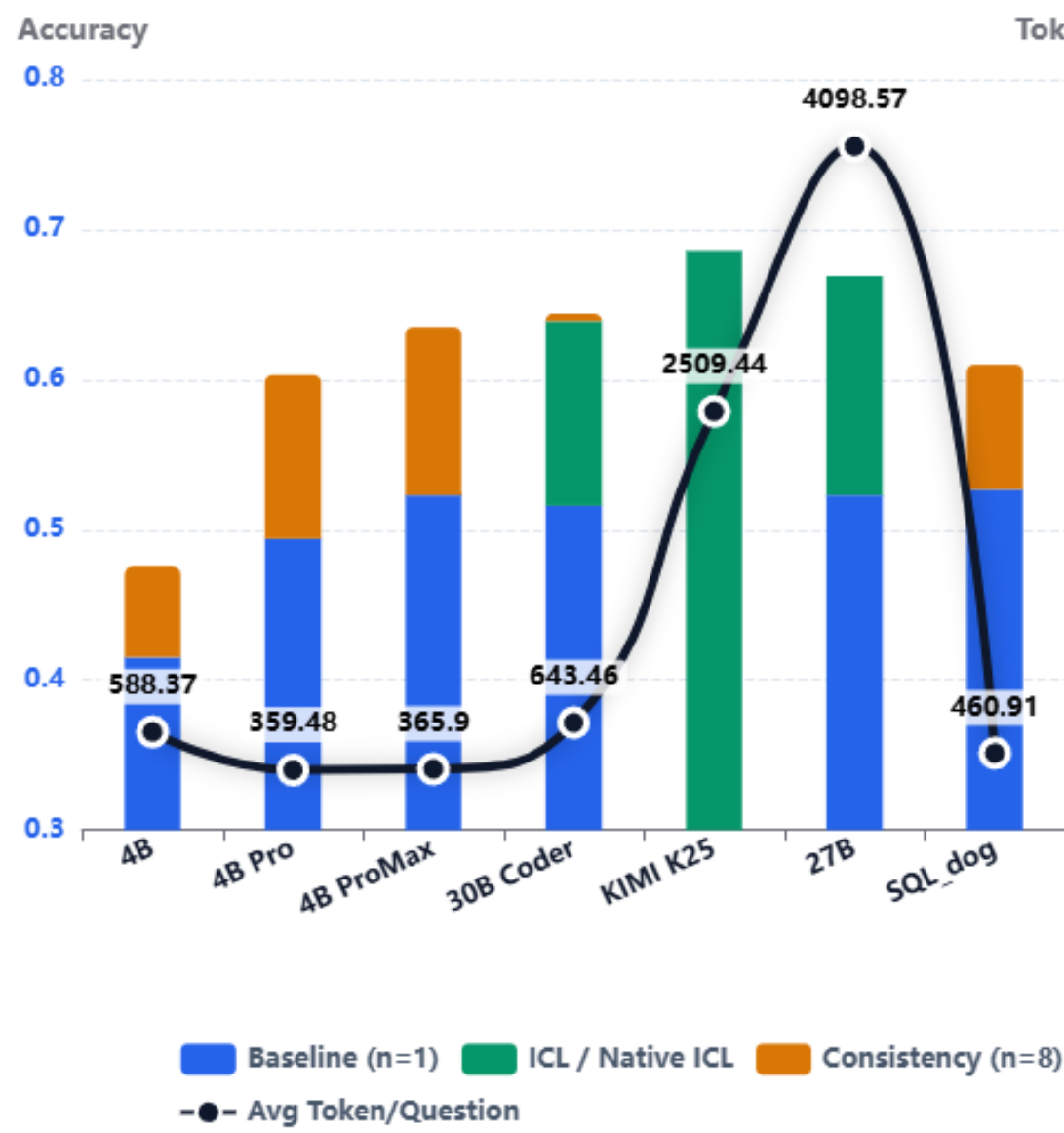
值检索

知识检索

SQL Dog



性能增量 vs. 推理成本

LEFT: A
RIGHT: TOKE

基础模型：Qwen/Qwen3-4B-Instruct-2507